

UNIT III - PYTHON FRAMEWORKS

Introduction to frameworks on Python–Flask and Django. Django: Creating web application –handle request and response–views and templates–Forms and generic views–SQLAlchemy.

What is a Framework? Definition:

- A framework is a set of tools and guidelines that help you build software applications more easily and efficiently.
- Think of it as a foundation or a template that you can build upon, which already has some common and useful features built-in.

Why Use a Framework?

- **Saves Time:** It provides pre-written code for common tasks, so you don't have to start from scratch.
- **Consistency:** It ensures that your code follows certain standards and practices, making it easier to maintain and understand.
- **Efficiency:** It helps you organize your code better and provides tools to handle repetitive tasks.

FLASK

- Flask is categorized as a micro framework because it does not depend on external libraries to perform the tasks of a framework.
- It has its tools, technologies, and libraries to support the functionalities of web application development.
- Since this framework is more independent and flexible, many developers prefer to start with Flask. Companies using Flask
- Definition:

Flask is a lightweight, micro web framework for Python designed to make web development simple and flexible.

Features:

1. **Lightweight:** It is a lightweight framework as it is independent of external libraries. It gives a quick start to the web development process of complex applications.

2. Independent: Flask gives independent or full control to the developer for creating applications. You can experiment with the architecture or the libraries of the framework.
3. Integrated Unit Testing: Flask's integrated unit testing system enables faster debugging, robust development, and freedom to experiment.
4. Secure Cookies: Secure cookie is an attribute of an HTTP request that enables the security of channels and ensures no unauthorized person has access to the text. Flask supports the feature of secure cookies.
5. Compatible: Flask is compatible with the latest technologies like Machine Learning, Cloud, etc.
6. Flexible and Scalable: Support WSGI templates that allow flexibility and scalability for web applications.
7. It comes with a built-in server and debugger.
8. Simple and adaptable configurations

When to Use Flask:

- Ideal for small to medium-sized web applications.
- Prototyping and developing APIs quickly.
- Projects that require high customization and flexibility.

Flask is used by the following giant companies:

1. Netflix
2. Airbnb
3. MIT
4. Reddit
5. Lyft

DJANGO

- Django is a python-based open-source framework to design web applications.
- It is a high-level web framework that is built to make the web development process faster and more efficient.
- Django is a free source with enhanced features and better performance.
- Developers choose Django for it enables them to use it for the standard functionalities with a limited interference of systems, protocols, and management.
- Django is also called a 'framework for fussy budgets with deadlines' as its framework encourages rapid development and clean, pragmatic design.
- The agile development process of the framework aims solely on providing quality with

rapidness and efficiency.

- Django deals with some of the basic development functions quickly like site maps, content organization, client information, and, so many more.
- It just focuses on finishing the application as quickly as possible.

Features:

- **"Batteries-Included"**: Comes with a wide array of built-in features.
- **ORM (Object-Relational Mapping)**: Simplifies database interactions.
- **Admin Interface**: Automatically generates a web-based admin interface.
- **Scalability**: Designed to handle high-traffic websites.
- **Security**: Includes built-in protections against common security threats.

When to Use Django:

- Ideal for large-scale applications.
- Projects that require a lot of built-in functionality out of the box.
- Developers who prefer convention over configuration.

Companies Using Django

Django is used by the following giant companies:

1. Instagram
2. Coursera
3. Mozilla
4. Pinterest
5. National Geographic
6. Spotify

DJANGO: CREATING WEB APPLICATION

Django is a high-level Python web framework that promotes rapid development. Django does this by automating many features so the user can focus on coding their app without worrying about aspects like security, views and much more.

Creating Your First Django Project:

1. Setting Up the Virtual Environment

Why Use a Virtual Environment?

A virtual environment allows you to manage dependencies for your project independently of other projects. This ensures that different projects can use different versions of the same packages without conflict.

Step 1: Install virtualenv

- Open your command prompt (Windows) or terminal (macOS/Linux).
- Install virtualenv if you haven't already:

```
pip install virtualenv
```

Step 2: Create a Virtual Environment

- Navigate to your project directory. If you don't have one, create it first:

```
mkdir todo_project
```

```
cd todo_project
```

- Create a virtual environment:

```
virtualenv venv
```

```
C:\Users\Shopika\AppData\Local\Python>mkdir todo_project

C:\Users\Shopika\AppData\Local\Python>cd todo_project

C:\Users\Shopika\AppData\Local\Python\todo_project>virtualenv venv
created virtual environment CPython3.12.4.final.0-64 in 16223ms
  creator CPython3Windows(dest=C:\Users\Shopika\AppData\Local\Python\todo_project\venv, clear=False, no_vcs_ignore=False, global=False)
  seeder FromAppData(download=False, pip=bundle, via=copy, app_data_dir=C:\Users\Shopika\AppData\Local\pypa\virtualenv)
    added seed packages: pip==24.1
  activators BashActivator,BatchActivator,FishActivator,NushellActivator,PowerShellActivator,PythonActivator

C:\Users\Shopika\AppData\Local\Python\todo_project>
```

Step 3: Activate the Virtual Environment

```
venv\Scripts\activate
```

```
C:\Users\Shopika\AppData\Local\Python\todo_project>venv\Scripts\activate

C:\Users\Shopika\AppData\Local\Python\todo_project>()

(venv) C:\Users\Shopika\AppData\Local\Python\todo_project>
```

You'll notice that your command prompt or terminal now shows (venv) at the beginning, indicating that the virtual environment is active.

Step 4: Installing Django

- With the virtual environment activated, install Django:

`pip install Django`

```
(venv) C:\Users\Shopika\AppData\Local\Python\todo_project>pip install django
Collecting django
  Downloading Django-5.0.7-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref<4,>=3.7.0 (from django)
  Using cached asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.3.1 (from django)
  Downloading sqlparse-0.5.1-py3-none-any.whl.metadata (3.9 kB)
Collecting tzdata (from django)
  Using cached tzdata-2024.1-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading Django-5.0.7-py3-none-any.whl (8.2 MB)
----- 8.2/8.2 MB 10.1 MB/s eta 0:00:00
Using cached asgiref-3.8.1-py3-none-any.whl (23 kB)
Downloading sqlparse-0.5.1-py3-none-any.whl (44 kB)
----- 44.2/44.2 kB 1.1 MB/s eta 0:00:00
Using cached tzdata-2024.1-py2.py3-none-any.whl (345 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.8.1 django-5.0.7 sqlparse-0.5.1 tzdata-2024.1

[notice] A new release of pip is available: 24.1 -> 24.2
[notice] To update, run: python.exe -m pip install --upgrade pip

(venv) C:\Users\Shopika\AppData\Local\Python\todo_project>
```

Step 5: Verify the Installation

- Check the installed Django version:

`django-admin --version`

```
(venv) C:\Users\Shopika\AppData\Local\Python\todo_project>django-admin --version
5.0.7
```

Step 6: Create a Django Project

- Use django-admin to create a new project.

```
django-admin startproject myproject
cd myproject
```

Project Structure:

myproject /

└── manage.py

```

|— myproject /
|   |— __init__.py
|   |— settings.py
|   |— urls.py
|   |— asgi.py
|   |— wsgi.py

```

Project Level

- **manage.py:** A tool that helps you run commands like starting the server and making changes to the database.
- **helloworld_project/__init__.py:** Just a file that tells Python this folder is important and part of the project.
- **helloworld_project/settings.py:** The main settings file where you configure things like which apps you are using and how your database is set up.
- **helloworld_project/urls.py:** The map that directs web addresses (URLs) to the right place in your code.
- **helloworld_project/asgi.py:** Helps your project talk to servers that handle web requests in a modern, asynchronous way.
- **helloworld_project/wsgi.py:** Helps your project talk to older, more traditional web servers.

NOTE:

- ❖ *ASGI stands for **Asynchronous Server Gateway Interface**. It is a specification that extends the capabilities of WSGI to support asynchronous web applications, allowing for handling long-lived connections like WebSockets and more efficient handling of concurrent requests.*
- ❖ *WSGI stands for **Web Server Gateway Interface**. It is a specification that defines how web servers communicate with web applications in Python, ensuring that web applications can work with various web servers seamlessly.*

Step 7: Start a Django App

- Inside the project directory, create a new app. Replace mypro with your app name:
python manage.py startapp mypro

App Structure:

```

mypro/
|— __init__.py

```

```

|— admin.py
|— apps.py
|— models.py
|— tests.py
|— views.py

```

App Level

- **helloworld_app/__init__.py**: Another file that tells Python this folder is important and part of the app.
- **helloworld_app/admin.py**: Where you set up what you can manage through Django's built-in admin interface.
- **helloworld_app/apps.py**: Contains settings specific to this particular app.
- **helloworld_app/models.py**: Where you define the structure of your data, like what kind of information you want to store in your database.
- **helloworld_app/tests.py**: Where you write tests to make sure your app works correctly.
- **helloworld_app/views.py**: Where you write the code that decides what happens when someone visits a web page.
- **helloworld_app/migrations/**: A folder where Django keeps track of changes you make to the database structure.

```

(venv) C:\Users\Shopika\AppData\Local\Python\todo_project\myproject>python manage.py startapp mypro
(venv) C:\Users\Shopika\AppData\Local\Python\todo_project\myproject>

```

> This PC > Local Disk (C:) > Users > Shopika > AppData > Local > Python > todo_project > myproject > mypro				
Name		Date modified	Type	Size
migrations		05-08-2024 22:16	File folder	
__init__		05-08-2024 22:16	Python File	0 KB
admin		05-08-2024 22:16	Python File	1 KB
apps		05-08-2024 22:16	Python File	1 KB
models		05-08-2024 22:16	Python File	1 KB
tests		05-08-2024 22:16	Python File	1 KB
views		05-08-2024 22:16	Python File	1 KB

Step 8: Run the Server

- Start the Django development server:

```
python manage.py runserver
```

- Visit <http://127.0.0.1:8000/> in your browser to see the to-do list application.

Explanation:

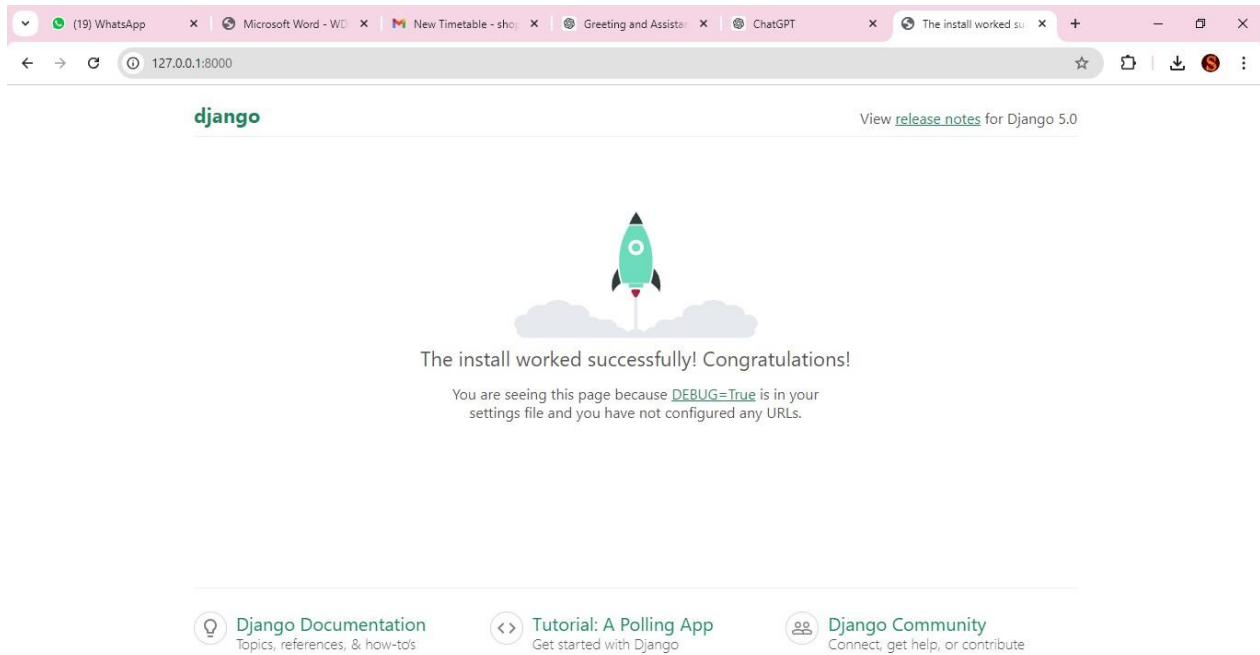
- The development server is a lightweight server provided by Django for development and testing purposes.

```
(venv) C:\Users\Shopika\AppData\Local\Python\todo_project\myproject>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations.
Run 'python manage.py migrate' to apply them.
August 05, 2024 - 22:20:04
Django version 5.0.7, using settings 'myproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

[05/Aug/2024 22:22:53] "GET / HTTP/1.1" 200 10629
Not Found: /favicon.ico
[05/Aug/2024 22:23:17] "GET /favicon.ico HTTP/1.1" 404 2113
```

Deactivate the Virtual Environment

In your command prompt or terminal where the virtual environment is activated, simply type:

deactivate

Confirm Deactivation:

After running the command, you should see that the `(venv)` prefix is no longer present in your command prompt or terminal, indicating that the virtual environment has been deactivated.

```
(venv) C:\Users\Shopika\AppData\Local\Python\todo_project\myproject>deactivate
C:\Users\Shopika\AppData\Local\Python\todo_project\myproject>
```

You've successfully downloaded Django, ran the development server and we're ready to start creating our app. Every time you start this project, you'll need to enter the `python manage.py runserver` into the command line to view the apps.

DJANGO: CREATING WEB APPLICATION

Django Project — This relates to the entire application and its parts

Django App — This refers to a sub-module of the project. An app is a web app that does something.

Creating a web application to **display Hello World and Current Date and Time** in Django involves several steps, from setting up your environment to deploying your application.

1. Set Up Your Environment

Ensure you have Python and pip installed. Then, install Django.

pip install django

2. Create a Django Project

Create a new Django project using the django-admin command.

django-admin startproject helloworld_project

This will create a directory named helloworld_project with the following structure:

helloworld_project/

manage.py

helloworld_project/

__init__.py

settings.py

urls.py

asgi.py

wsgi.py

3. Create a Django App

Inside your project, create a new app.

cd helloworld_project

python manage.py startapp helloworld_app

The helloworld_app directory will have this structure:

helloworld_app/

__init__.py

admin.py

apps.py

models.py

tests.py

views.py

migrations/

4. Configure the App

Add your new app to the INSTALLED_APPS list in **helloworld_project/settings.py**:

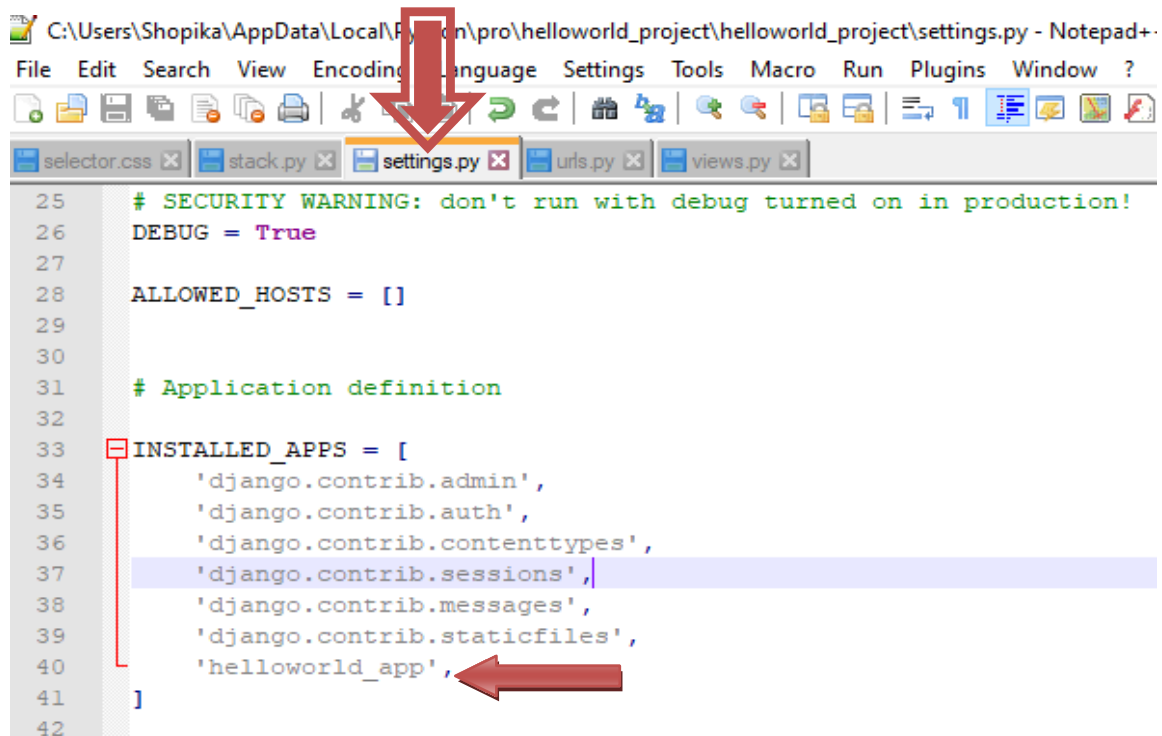
```
# helloworld_project/settings.py
```

```
INSTALLED_APPS = [
```

```
...
```

```
'helloworld_app',
```

```
]
```



5. Create a View

In **helloworld_app/views.py**, create a view that returns "Hello, World!" and the current date and time:

```
# helloworld_app/views.py
```

```
from django.http import HttpResponse
```

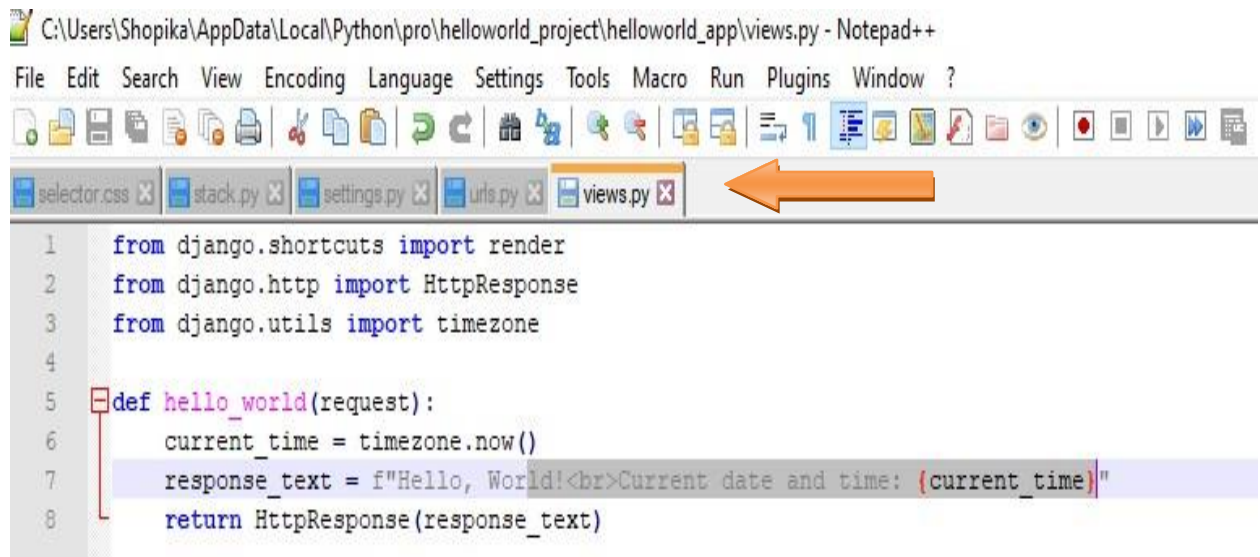
```
from django.utils import timezone
```

```
def hello_world(request):
```

```
    current_time = timezone.now()
```

```
    response_text = f"Hello, World!<br>Current date and time: {current_time}"
```

```
    return HttpResponse(response_text)
```



```
C:\Users\Shopika\AppData\Local\Python\pro\helloworld_project\helloworld_app\views.py - Notepad++
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
selector.css x stack.py x settings.py x urls.py x views.py x
1 from django.shortcuts import render
2 from django.http import HttpResponse
3 from django.utils import timezone
4
5 def hello_world(request):
6     current_time = timezone.now()
7     response_text = f"Hello, World!<br>Current date and time: {current_time}"
8     return HttpResponse(response_text)
```

6. Configure URLs

Map a URL to your view. Edit **helloworld_project/urls.py**:

helloworld_project/urls.py

from django.contrib import admin

from django.urls import path

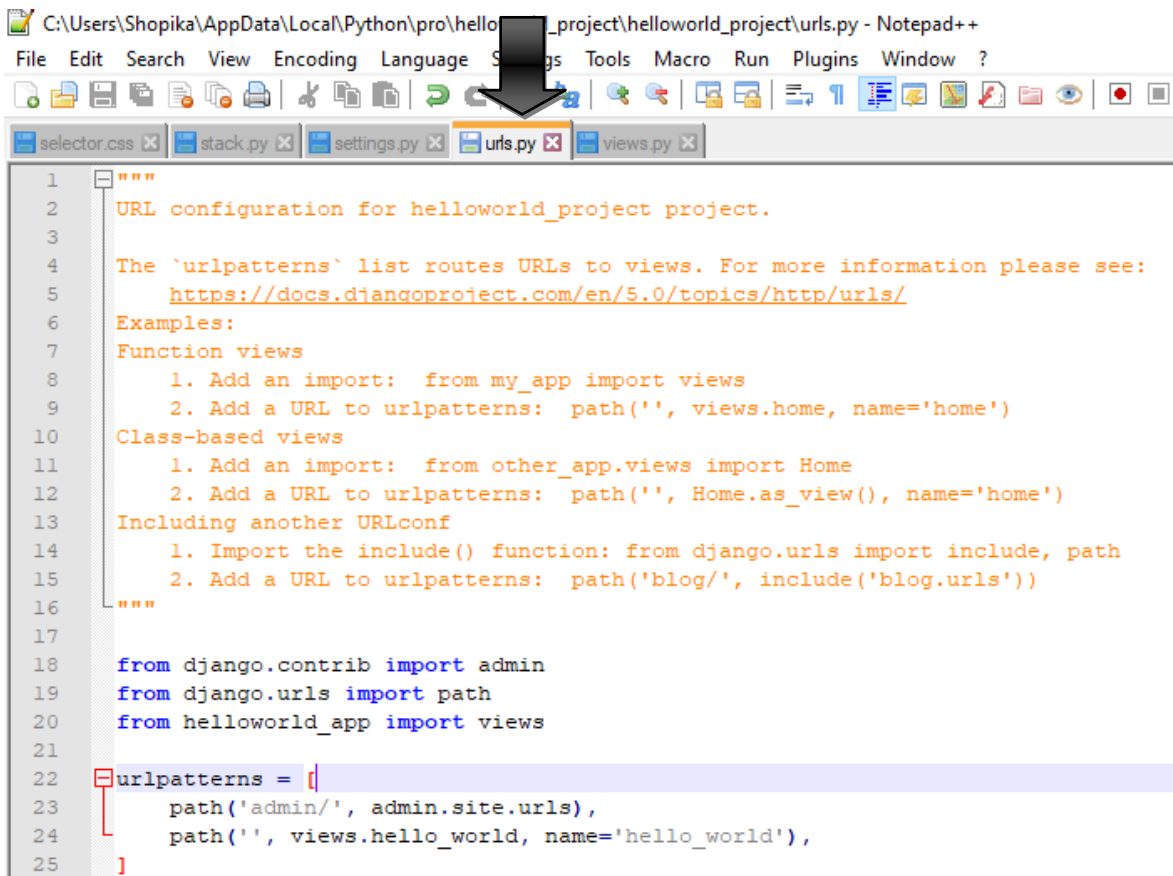
from helloworld_app import views

urlpatterns = [

 path('admin/', admin.site.urls),

 path("", views.hello_world, name='hello_world'),

]



```

1  """
2  URL configuration for helloworld_project project.
3
4  The 'urlpatterns' list routes URLs to views. For more information please see:
5      https://docs.djangoproject.com/en/5.0/topics/http/urls/
6  Examples:
7  Function views
8      1. Add an import:  from my_app import views
9      2. Add a URL to urlpatterns:  path('', views.home, name='home')
10 Class-based views
11     1. Add an import:  from other_app.views import Home
12     2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
13 Including another URLconf
14     1. Import the include() function: from django.urls import include, path
15     2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
16 """
17
18 from django.contrib import admin
19 from django.urls import path
20 from helloworld_app import views
21
22 urlpatterns = [
23     path('admin/', admin.site.urls),
24     path('', views.hello_world, name='hello_world'),
25 ]

```

7. Run the Development Server

Start the Django development server:

```
python manage.py runserver
```

```

(venv) C:\Users\Shopika\AppData\Local\Python\pro\helloworld_project>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations f
or app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
August 07, 2024 - 09:14:42
Django version 5.0.7, using settings 'helloworld_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

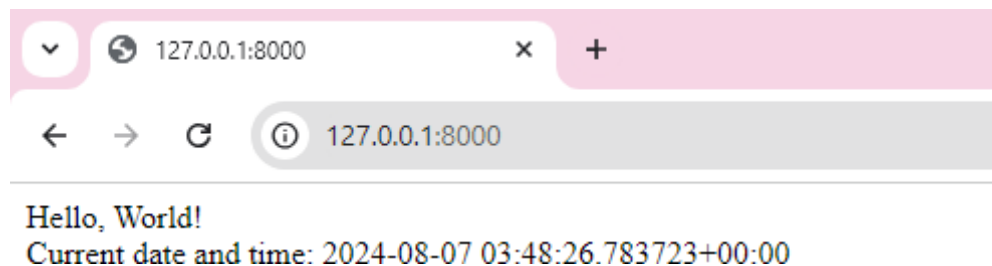
[07/Aug/2024 09:14:53] "GET / HTTP/1.1" 200 13
Not Found: /favicon.ico
[07/Aug/2024 09:14:54] "GET /favicon.ico HTTP/1.1" 404 2242
C:\Users\Shopika\AppData\Local\Python\pro\helloworld_project\helloworld_app\views.py changed, reloading

```

8. Access the Application

Navigate to <http://127.0.0.1:8000/> in your web browser.

You should see "Hello, World!" along with the current date and time.



DJANGO: HANDLE REQUEST AND RESPONSE

The client-server architecture includes two major components request and response. The Django framework uses client-server architecture to implement web applications. When a client requests for a resource, an HttpRequest object is created and correspond view function is called that returns HttpResponse object. To handle request and response, Django provides HttpRequest and HttpResponse classes. Each class has its own attributes and methods.

DJANGO HTTP REQUEST

This class is defined in the django.http module and used to handle the client request. Following are the attributes of this class.

Django HttpRequest Attributes

Attribute	Description
HttpRequest.scheme	A string representing the scheme of the request (HTTP or HTTPS usually).
HttpRequest.body	It returns the raw HTTP request body as a byte string.
HttpRequest.path	It returns the full path to the requested page does not include the scheme or domain.
HttpRequest.path_info	It shows path info portion of the path.
HttpRequest.method	It shows the HTTP method used in the request.
HttpRequest.encoding	It shows the current encoding used to decode form submission data.

HttpRequest Methods – Django

Attribute	Description
HttpRequest.get_host()	It returns the original host of the request.
HttpRequest.get_port()	It returns the originating port of the request.
HttpRequest.get_full_path()	It returns the path, plus an appended query string, if applicable.
HttpRequest.build_absolute_uri (<i>location</i>)	It returns the absolute URI form of location.

DJANGO HTTPRESPONSE

This class is a part of `django.http` module. It is responsible for generating response corresponds to the request and back to the client.

HttpResponse Attributes – Django

Attribute	Description
<code>HttpResponse.content</code>	A bytestring representing the content, encoded from a string if necessary.
<code>HttpResponse.charset</code>	It is a string denoting the charset in which the response will be encoded.
<code>HttpResponse.status_code</code>	It is an HTTP status code for the response.
<code>HttpResponse.reason_phrase</code>	The HTTP reason phrase for the response.
<code>HttpResponse.streaming</code>	It is false by default.
<code>HttpResponse.closed</code>	It is True if the response has been closed.

HttpResponse Methods – Django

Method	Description
<code>HttpResponse.__init__(content="", content_type=None, status=200, reason=None, charset=None)</code>	It is used to instantiate an <code>HttpResponse</code> object with the given page content and content type.
<code>HttpResponse.__setitem__(header, value)</code>	It is used to set the given header name to the given value.
<code>HttpResponse.__delitem__(header)</code>	It deletes the header with the given name.
<code>HttpResponse.__getitem__(header)</code>	It returns the value for the given header name.
<code>HttpResponse.has_header(header)</code>	It returns either True or False based on a case-insensitive check for a header with the provided name.

DJANGO REQUEST-RESPONSE CYCLE

1. Request Handling Process

a. URL Configuration (Routing)

When a user makes a request to a Django application (e.g., by entering a URL in a browser), Django uses the URL configuration (URLconf) to determine which view should handle the request.

- **urls.py:** This file maps URLs to views. When a request is received, Django checks the URL patterns defined in `urls.py` to find the corresponding view function.

```
# helloworld_project/urls.py
```



```

from django.contrib import admin
from django.urls import path
from helloworld_app import views
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", views.hello_world, name='hello_world'),
]

```

b. The View

The view is a Python function or class-based view that takes the request as an argument and returns a response. It contains the logic to process the request and generate the appropriate response.

- **views.py:** The view function processes the request data, interacts with the model if necessary, and returns a response.

```

# helloworld_app/views.py
from django.http import HttpResponse
from django.utils import timezone
def hello_world(request):
    current_time = timezone.now()
    response_text = f"Hello, World!<br>Current date and time: {current_time}"
    return HttpResponse(response_text)

```

2. Request Object

When Django receives a request, it creates an `HttpRequest` object that contains metadata about the request, such as the HTTP method, headers, and user information.

- **HttpRequest:** The request object provides information about the request, including data submitted via GET or POST methods, cookies, session data, and more.

Example usage in a view function

```

def hello_world(request):
    user_agent = request.META['HTTP_USER_AGENT'] # Retrieve the user's browser
    information
    current_time = timezone.now()

    response_text = f"Hello, World!<br>Current date and time: {current_time}<br>User Agent:
    {user_agent}"
    return HttpResponse(response_text)

```


3. Response Object

The view processes the request and returns an `HttpResponse` object. The `HttpResponse` object contains the content that will be sent back to the user's browser.

- **HttpResponse:** The response object is used to define the content, MIME type, and status code of the response.

Example usage in a view function

```
def hello_world(request):
```

```
    current_time = timezone.now()
```

```
    response_text = f"Hello, World!<br>Current date and time: {current_time}"
```

```
    return HttpResponse(response_text, content_type="text/html", status=200)
```

4. Middleware

Before the request reaches the view and before the response is returned to the browser, it passes through various middleware components. Middleware can process the request or response globally.

- **Middleware:** Middleware is a way to process requests globally before they reach the view or after the view has processed them.

Example of middleware in settings.py

```
MIDDLEWARE = [
```

```
    'django.middleware.security.SecurityMiddleware',
```

```
    'django.contrib.sessions.middleware.SessionMiddleware',
```

```
    'django.middleware.common.CommonMiddleware',
```

```
    'django.middleware.csrf.CsrfViewMiddleware',
```

```
    'django.contrib.auth.middleware.AuthenticationMiddleware',
```

```
    'django.contrib.messages.middleware.MessageMiddleware',
```

```
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
```

```
]
```

4. Running the Server

Start the Django development server to see the application in action.

```
python manage.py runserver
```

5. Accessing the Application

Navigate to `http://127.0.0.1:8000/` in your web browser to see the output:

OUTPUT:

Hello, World!

Current date and time: YYYY-MM-DD HH:MM:SS

User Agent: [Your Browser's User Agent]

- **Request:** The HttpRequest object is created, containing metadata about the request.
- **URL Routing:** The URLconf maps the URL to the corresponding view.
- **View:** The view processes the request and returns an HttpResponse object.
- **Response:** The HttpResponse object contains the content to be sent back to the browser.
- **Middleware:** Middleware processes the request before it reaches the view and the response before it is sent back to the browser.

DJANGO VIEWS

- ❖ In Django, a view is a Python function or class that takes a web request and returns a web response. This response could be an HTML page, a redirect, a 404 error, an XML document, an image, or anything else that Django can generate.
- ❖ Django views are part of the user interface — they usually render the HTML/CSS/Javascript in your Template files into what you see in your browser when you render a web page.
- ❖ In Django, a view works as:
 - **Request:** You ask for something by entering a URL.
 - **Processing:** The view figures out what you're asking for and gathers the necessary information.
 - **Response:** The view sends that information back to you, often as an HTML web page that you see in your browser.
- ❖ A web page that uses Django is full of views with different tasks and missions.
- ❖ Views are usually put in a file called views.py located on your app's folder.

Types of Views:

1. Function-Based Views (FBV):

- ✓ Traditional way to define views using functions.
- ✓ Each view is a function that takes a request object and returns a response.
- ✓ FBVs are simple to write and understand, especially for those who are new to Django or web development.

Structure of a Function-Based View

- An FBV is typically a function that takes at least one argument, which is usually the request object.
- The function then processes the request, performs any necessary operations (like querying the database), and returns a response (often an HTML page).

Example of a Simple Function-Based View

```
from django.http import HttpResponse
def hello_world(request):
    return HttpResponse("Hello, World!")
```

Advantages of Function-Based Views

- **Simplicity:** FBVs are straightforward and easy to write, especially for small, simple views.
- **Flexibility:** You have complete control over how the request is handled, as there are no predefined methods or structures you have to follow.
- **Explicitness:** Every action is explicitly coded, making it clear what the view is doing.

When to Use Function-Based Views

- Use FBVs when your views are simple and don't require much structure or reuse of code.
- They are ideal for small projects or when handling simple tasks like rendering a template or returning a basic HTTP response.

2. Class-Based Views (CBV):

- ❖ More advanced way to define views using Python classes.
- ❖ Allows reuse of common patterns and provides more structure and functionality through inheritance
- ❖ **Definition:** Class-Based Views (CBVs) are Django views organized as Python classes rather than functions. Each class represents a view, and methods within the class handle specific HTTP request methods (like GET, POST, etc.).
- ❖ **Purpose:** The main purpose of CBVs is to promote the reuse of common patterns, making your code more modular, organized, and easier to maintain.

BASIC STRUCTURE OF A CLASS-BASED VIEW

- A basic CBV typically extends Django's View class, and defines methods to handle different HTTP methods.

Example:

```
from django.views import View
from django.http import HttpResponse

class MyView(View):
    def get(self, request):
        return HttpResponse("This is a GET request")
```

```
def post(self, request):  
    return HttpResponseRedirect("This is a POST request")
```

Here, MyView handles both GET and POST requests by defining corresponding methods (get and post).

Commonly Used Class-Based Views

Django provides several base classes and generic views to handle common web development tasks. These are part of Django's Generic Class-Based Views (GCBVs).

a. Base View Class

- **View Class:** The most basic class you can extend to create custom views.
- You need to define methods like get(), post(), etc., depending on the type of request you want to handle.

Example:

```
from django.views import View  
class SimpleView(View):  
    def get(self, request):  
        return HttpResponseRedirect("Simple GET request handled")
```

b. Generic Class-Based Views

Generic views are pre-built views that handle common patterns like displaying a list of objects, showing details of an object, creating, updating, and deleting objects.

1. ListView:

- Used to display a list of objects from a database model.

Example:

```
from django.views.generic import ListView  
from .models import Article  
class ArticleListView(ListView):  
    model = Article  
    template_name = 'articles/article_list.html'
```

This view automatically queries the Article model and passes the list of articles to the template article_list.html.

2. DetailView:

- Used to display a single object based on a primary key or slug.

Example:

```
from django.views.generic import DetailView
from .models import Article
class ArticleDetailView(DetailView):
    model = Article
    template_name = 'articles/article_detail.html'
```

This view fetches a specific article and displays its details using the article_detail.html template.

3. *CreateView:*

- Handles the creation of a new object in the database.

Example:

```
from django.views.generic import CreateView
from .models import Article
from django.urls import reverse_lazy
class ArticleCreateView(CreateView):
    model = Article
    fields = ['title', 'content']
    template_name = 'articles/article_form.html'
    success_url = reverse_lazy('article-list')
```

This view provides a form for creating a new article and redirects to the article list upon successful submission.

4. *UpdateView:*

- Allows editing of an existing object.

Example:

```
from django.views.generic import UpdateView
from .models import Article
from django.urls import reverse_lazy
class ArticleUpdateView(UpdateView):
    model = Article
    fields = ['title', 'content']
    template_name = 'articles/article_form.html'
    success_url = reverse_lazy('article-list')
```

5. *DeleteView:*

- Handles the deletion of an object.

Example:

```
from django.views.generic import DeleteView
from .models import Article
from django.urls import reverse_lazy
class ArticleDeleteView(DeleteView):
    model = Article
    template_name = 'articles/article_confirm_delete.html'
    success_url = reverse_lazy('article-list')
```

Advantages of Class-Based Views

- **Reusability:** CBVs allow you to create reusable components, reducing code duplication.
- **Extensibility:** You can easily extend existing views by overriding methods or using mixins.
- **Organization:** CBVs promote a more structured and organized codebase, especially for larger projects.
- **Built-in Functionality:** Generic views provide out-of-the-box solutions for common tasks, saving development time.

When to Use Class-Based Views

- **Complex Views:** Use CBVs for complex views that involve multiple actions, data processing, or reuse of logic.
- **Built-In Generic Views:** Use CBVs when you can leverage Django's generic views to handle common patterns like list views, detail views, and forms.
- **Large Projects:** In large projects with many views, CBVs help maintain a clean and organized code structure.

Class-Based Views in Django offer a powerful and flexible way to handle complex views and repetitive tasks

Comparison: Function-Based Views vs. Class-Based Views

Aspect	Function-Based Views (FBVs)	Class-Based Views (CBVs)
Simplicity	Simple and straightforward	More complex due to class and method structure
Control	Full control over request handling	Can override methods for specific behaviour
Reusability	Less reusable, requires manual code duplication	Highly reusable through inheritance and mixins

Flexibility	Highly flexible, no constraints	Flexible but within the confines of class structure
Learning Curve	Easier for beginners	Steeper learning curve, especially for new developers
Best Use Cases	Simple, one-off views	Complex views, repetitive patterns, and large projects

DJANGO TEMPLATES

- Django provides a convenient way to generate dynamic HTML pages by using its template system.
- A template consists of static parts of the desired HTML output as well as some special syntax describing how dynamic content will be inserted.
- Django template engine is used to separate the design from the python code and allows us to build dynamic web pages.
- **Definition:** A template in Django is a text file that defines the structure or layout of an HTML page, which includes placeholders for dynamic content.
- **Purpose:** Templates are used to separate the presentation layer (HTML) from the business logic in a web application. This ensures that the design and content management of a web page are separated from the underlying code.

Key Features of Django Templates:

- **Dynamic Content Rendering:** Templates allow you to generate dynamic HTML by inserting data from Django views into placeholders.
- **Template Language:** Django has its own template language that includes variables, tags, filters, and comments to help you manage content dynamically.
- **Inheritance:** Templates can inherit from other templates, allowing you to create a base layout that can be extended by other templates, promoting DRY (Don't Repeat Yourself) principles.

DJANGO TEMPLATE LANGUAGE COMPONENTS

a. Template Variables

- **Definition:** Variables are placeholders in templates that are replaced with values when the template is rendered.
- **Syntax:** Variables are enclosed within double curly braces {{ }}.

Example:

```
<h1>Welcome, {{ username }}!</h1>
```


Here, {{ username }} will be replaced with the actual username provided by the view.

b. Template Tags

- **Definition:** Tags are used for logic in templates, such as loops, conditionals, and including other templates. Tags are enclosed within {% %}.
- **Common Tags:**
 - **{% for %}:** Used for looping over a list.
 - **{% if %}:** Used for conditional statements.
 - **{% block %} and {% extends %}:** Used for template inheritance.

Example:

```
{% if user.is_authenticated %}
```

```
<p>Hello, {{ user.username }}!</p>
```

```
{% else %}
```

```
<p>Hello, Guest!</p>
```

```
{% endif %}
```

SAMPLE FOR FOR LOOP

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Item List</title>
```

```
</head>
```

```
<body>
```

```
<h1>Items</h1>
```

```
<ul>
```

```
{% for item in items %}
```

```
<li>
```

```
{% if forloop.first %}
```

```
<strong>First item:</strong>
```

```
{% endif %}
```

```
{{ forloop.counter }}: {{ item.name }}
```

```
{% if forloop.last %}
```

```
<br><em>Last item!</em>
```

```
{% endif %}
```

```
</li>
```

```

        {% endfor %}
    </ul>
</body>
</html>

```

c. Template Filters

- **Definition:** Filters are used to modify the display of variables, like formatting dates, converting text to uppercase, etc.
- **Syntax:** Filters are applied using a pipe |.

Example:

```
<p>The current date is {{ date_today|date:"F d, Y" }}</p>
```

Here, the date filter formats the date_today variable to "Month Day, Year" format.

d. Template Comments

- **Definition:** Comments in Django templates are ignored by the template engine and are not rendered in the final HTML.
- **Syntax:** Comments are enclosed within {# #}.

Example:

```
{# This is a comment and will not appear in the HTML #}
```

TEMPLATE INHERITANCE

a. Purpose of Template Inheritance

- **Definition:** Template inheritance allows you to define a base template (e.g., for the layout of a website) and then extend it in other templates.
- **Advantages:** This approach promotes reusability and consistency across the website.

b. Base Template

- A base template contains common elements like the header, footer, and navigation bar, which will be shared across multiple pages.

Example: base.html

```

<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}My Site{% endblock %}</title>
</head>
<body>
    <header>

```

```
<h1>Welcome to My Site</h1>
</header>
<nav>
    <!-- Navigation menu -->
</nav>
<main>
    {% block content %}{% endblock %}
</main>
<footer>
    <p>&copy; 2024 My Site</p>
</footer>
</body>
</html>
```

c. Extending a Base Template

- Child templates can extend the base template and replace the content of the blocks defined in the base template.

Example: home.html

```
{% extends "base.html" %}
{% block title %}Home - My Site{% endblock %}
{% block content %}
<h2>Home Page</h2>
<p>This is the home page content.</p>
{% endblock %}
```

d. Block Tag

- Purpose:** The `{% block %}` tag is used to define areas in a template that can be overridden by child templates.
- Syntax:**
`{% block block_name %}Default content{% endblock %}`

TEMPLATE CONTEXT

- Definition:** The context in Django refers to a dictionary of data that is passed from the view to the template. This data can include variables, lists, or any other data needed to render the template.
- Rendering a Template:** The render function in Django combines the template with the context and returns an HTTP response with the rendered HTML.

Example:

```
def my_view(request):  
    context = {  
        'username': 'JohnDoe',  
        'date_today': datetime.date.today(),  
    }  
    return render(request, 'my_template.html', context)
```

- **Accessing Context in Templates:** In the template, you can access context variables using `{{ variable_name }}`.

Example:

```
<p>Username: {{ username }}</p>  
<p>Today's Date: {{ date_today }}</p>
```

TEMPLATE LOADERS

- **Definition:** Template loaders are responsible for finding and loading templates from the file system or other sources like databases.
- *Default Loaders:*
 - ✓ **Filesystem Loader:** Loads templates from the directories specified in `TEMPLATES['DIRS']`.
 - ✓ **App Directories Loader:** Looks for templates within each app's templates directory.

TEMPLATE INCLUSIONS

- **Purpose:** The `{% include %}` tag allows you to include the content of another template within a template.

Example:

```
{% include "partials/header.html" %}  
<p>Main content here.</p>  
{% include "partials/footer.html" %}
```

DJANGO FORMS

- A Django form class is a powerful tool for creating and managing forms in web applications.
- Django provides a Form class which is used to create HTML forms. It describes a form and how it works and appears.
- It allows you to define form fields, apply validation rules, and handle user input in a structured and reusable way.
- Forms are used for taking input from the user in some manner and using that information

for logical operations on databases. For example, registering a user by taking input such as his name, email, password, etc.

- Django maps the fields defined in Django forms into HTML input fields.
- Django handles three distinct parts of the work involved in forms:
 - Preparing and restructuring data to make it ready for rendering.
 - Creating HTML forms for the data.
 - Receiving and processing submitted forms and data from the client.

Why Use Django Forms?

1. **Organize Information:** Forms help you collect and organize user input in a structured way.
2. **Validate Data:** They check if the information entered by users is correct (like making sure an email address is properly formatted).
3. **Display Data:** Forms let you show the fields (like text boxes and drop-down menus) on your website in a clean and consistent manner.

The most important part of a form and the only required part is the list of fields it defines. Fields are specified by class attributes.

Syntax:

Django Fields work like Django Model Fields and have the syntax:
`field_name = forms.FieldType(**options)`

Here is a list of all Form Field types used in Django

****kwargs** : Allows you to pass a variable number of keyword arguments to a function.

Name	Class	HTML Input
CharField	class CharField(**kwargs)	TextInput
ChoiceField	class ChoiceField(**kwargs)	Select
DateField	class DateField(**kwargs)	DateInput
EmailField	class EmailField(**kwargs)	EmailInput
FileField	class FileField(**kwargs)	ClearableFileInput
ImageField	class ImageField(**kwargs)	ClearableFileInput
IntegerField	class IntegerField(**kwargs)	NumberInput when Field.localize is False, else TextInput
MultipleChoiceField	class MultipleChoiceField(**kwargs)	SelectMultiple
URLField	class URLField(**kwargs)	URLInput
UUIDField	class UUIDField(**kwargs)	TextInput

Creating Forms in Django

- Creating a form in Django is completely similar to creating a model, one needs to specify

what fields would exist in the form and of what type.

- Form is a class in Django used to create and handle forms that are not directly tied to database models. It is ideal for simpler forms that do not need the full power of Django's model forms.

For example, to input, a registration form one might need First Name (CharField), Roll Number (IntegerField), and so on.

Syntax:

```
from django import forms
```

```
class FormName(forms.Form):
```

```
    # Each field would be mapped as an input field in HTML
```

```
    field_name = forms.Field(**options)
```

EXAMPLE:

```
from django import forms
```

```
    # creating a form
```

```
class InputForm(forms.Form):
```

```
    first_name = forms.CharField(max_length = 200)
```

```
    last_name = forms.CharField(max_length = 200)
```

```
    roll_number = forms.IntegerField( help_text = "Enter 6 digit roll number" )
```

```
    password = forms.CharField(widget = forms.PasswordInput())
```

Render Django Forms

Django form fields have several built-in methods to ease the work of the developer but sometimes one needs to implement things manually for customizing User Interface(UI). A form comes with 3 in-built methods that can be used to render Django form fields.

1. {{ form.as_table }} will render them as table cells wrapped in <tr> tags
2. {{ form.as_p }} will render them wrapped in <p> tags
3. {{ form.as_ul }} will render them wrapped in tags

To render this form into a view, move to views.py and create a home_view as below.

```
from django.shortcuts import render
```

```
from .forms import InputForm
```

```
# Create your views here.
```

```
def home_view(request):
```

```
    context = {}
```

```
    context['form']= InputForm()
```

```
    return render(request, "home.html", context)
```

Now create an instance of the form class created above in forms.py.

Now let's edit templates > home.html

```
<form action = "" method = "post">
    {% csrf_token %}
    {{ form }}
    <input type="submit" value=Submit">
</form>
```

Now, Run the development server: `python manage.py runserver`

Open your web browser and go to **http://127.0.0.1:8000/home** to view the registration form

VALIDATING FORM DATA IN DJANGO

a. Built-in Validation

Django's `forms.Form` includes several built-in validation mechanisms for form fields. These mechanisms ensure that the data entered by the user adheres to the required format and constraints.

1. Field-Level Validation:

CharField: Automatically validates that the input is a string and checks for maximum length if specified.

EmailField: Checks that the input is a valid email address.

IntegerField: Validates that the input is an integer and can enforce minimum and maximum values.

DateField: Ensures the input is a valid date.

Examples:

```
# forms.py
```

```
from django import forms
```

```
class UserForm(forms.Form):
```

```
    username = forms.CharField(max_length=150) # Validates string length
```

```
    email = forms.EmailField() # Validates email format
```

```
    age = forms.IntegerField(min_value=18, max_value=120) # Validates integer range
```

```
    birth_date = forms.DateField() # Validates date format
```

2. Required Fields: By default, fields are required unless specified otherwise. To make a field optional, set `required=False`.

Example:

```
# forms.py
```

```
class OptionalForm(forms.Form):
```

```
    comment = forms.CharField(required=False) # Comment is optional
```

3. Regex Validation: Use RegexField to validate input against a regular expression pattern.

Example:

```
# forms.py
```

```
class RegexForm(forms.Form):
```

```
    phone_number = forms.RegexField(
```

```
        regex=r'^\d{10}$',
```

```
        error_messages={'invalid': 'Enter a valid 10-digit phone number.'})
```

```
)
```

b. Custom Validation

Django allows you to add custom validation logic by defining methods in your form class. This is useful when you need to enforce more complex rules that aren't covered by built-in validators.

Field-Specific Validation: Define a method named `clean_<fieldname>` where `<fieldname>` is the name of the field you want to validate. This method is called automatically during form validation.

Example:

```
# forms.py
```

```
class CustomValidationForm(forms.Form):
```

```
    email = forms.EmailField()
```

```
    def clean_email(self):
```

```
        email = self.cleaned_data.get('email')
```

```
        if email.endswith('@example.com'):
```

```
            raise forms.ValidationError("Emails from example.com are not allowed.")
```

```
        return email
```

DJANGO GENERIC VIEWS

Introduction to Django Generic Views What Are Generic Views?

- **Definition:** Django Generic Views are pre-built class-based views provided by Django

that simplify common web development tasks such as displaying a list of objects, handling forms, and performing CRUD (Create, Read, Update, Delete) operations.

- **Purpose:** They help reduce boilerplate code and promote code reuse, making Django applications easier to develop and maintain.

Why Use Generic Views?

- **Code Efficiency:** By leveraging generic views, developers can avoid writing repetitive code for common tasks.
- **Maintainability:** Generic views lead to cleaner, more maintainable code.
- **Flexibility:** Although generic views provide default behaviors, they can be customized to fit specific application needs.

DJANGO VIEWS

- ❖ In Django, a view is a Python function or class that takes a web request and returns a web response. This response could be an HTML page, a redirect, a 404 error, an XML document, an image, or anything else that Django can generate.
- ❖ Django views are part of the user interface — they usually render the HTML/CSS/Javascript in your Template files into what you see in your browser when you render a web page.
- ❖ In Django, a view works as:
 - **Request:** You ask for something by entering a URL.
 - **Processing:** The view figures out what you're asking for and gathers the necessary information.
 - **Response:** The view sends that information back to you, often as an HTML web page that you see in your browser.
- ❖ A web page that uses Django is full of views with different tasks and missions.
- ❖ Views are usually put in a file called views.py located on your app's folder.

Types of Views:

3. **Function-Based Views (FBV):**

- ✓ Traditional way to define views using functions.
- ✓ Each view is a function that takes a request object and returns a response.
- ✓ FBVs are simple to write and understand, especially for those who are new to Django or web development.

Structure of a Function-Based View

- An FBV is typically a function that takes at least one argument, which is usually the request object.
- The function then processes the request, performs any necessary operations (like querying the database), and returns a response (often an HTML page).

Example of a Simple Function-Based View

```
from django.http import HttpResponse  
def hello_world(request):  
    return HttpResponse("Hello, World!")
```

Advantages of Function-Based Views

- **Flexibility:** You have complete control over how the request is handled, as there are no predefined methods or structures you have to follow.
- **Explicitness:** Every action is explicitly coded, making it clear what the view is doing.

When to Use Function-Based Views

- Use FBVs when your views are simple and don't require much structure or reuse of code.
- They are ideal for small projects or when handling simple tasks like rendering a template or returning a basic HTTP response.

4. Class-Based Views (CBV):

- ❖ More advanced way to define views using Python classes.
- ❖ Allows reuse of common patterns and provides more structure and functionality through inheritance
- ❖ **Definition:** Class-Based Views (CBVs) are Django views organized as Python classes rather than functions. Each class represents a view, and methods within the class handle specific HTTP request methods (like GET, POST, etc.).
- ❖ **Purpose:** The main purpose of CBVs is to promote the reuse of common patterns, making your code more modular, organized, and easier to maintain.

BASIC STRUCTURE OF A CLASS-BASED VIEW

- A basic CBV typically extends Django's View class, and defines methods to handle different HTTP methods.

Example:

```
from django.views import View  
from django.http import HttpResponse  
class MyView(View):
```

```
def get(self, request):  
    return HttpResponse("This is a GET request")  
  
def post(self, request):  
    return HttpResponse("This is a POST request")
```

Here, MyView handles both GET and POST requests by defining corresponding methods (get and post).

Commonly Used Class-Based Views

Django provides several base classes and generic views to handle common web development tasks. These are part of Django's Generic Class-Based Views (GCBVs).

a. Base View Class

- **View Class:** The most basic class you can extend to create custom views.
- You need to define methods like get(), post(), etc., depending on the type of request you want to handle.

Example:

```
from django.views import View  
  
class SimpleView(View):  
    def get(self, request):  
        return HttpResponse("Simple GET request handled")
```

b. Generic Class-Based Views

Generic views are pre-built views that handle common patterns like displaying a list of objects, showing details of an object, creating, updating, and deleting objects.

6. *ListView:*

- Used to display a list of objects from a database model.

Example:

```
from django.views.generic import ListView  
from .models import Article  
  
class ArticleListView(ListView):  
    model = Article  
    template_name = 'articles/article_list.html'
```

This view automatically queries the Article model and passes the list of articles to the template article_list.html.

7. *DetailView:*

- Used to display a single object based on a primary key or slug.

Example:

```
from django.views.generic import DetailView
from .models import Article
class ArticleDetailView(DetailView):
    model = Article
    template_name = 'articles/article_detail.html'
```

This view fetches a specific article and displays its details using the article_detail.html template.

8. *CreateView:*

- Handles the creation of a new object in the database.

Example:

```
from django.views.generic import CreateView
from .models import Article
from django.urls import reverse_lazy
class ArticleCreateView(CreateView):
    model = Article
    fields = ['title', 'content']
    template_name = 'articles/article_form.html'
    success_url = reverse_lazy('article-list')
```

This view provides a form for creating a new article and redirects to the article list upon successful submission.

9. *UpdateView:*

- Allows editing of an existing object.

Example:

```
from django.views.generic import UpdateView
from .models import Article
from django.urls import reverse_lazy
class ArticleUpdateView(UpdateView):
    model = Article
    fields = ['title', 'content']
    template_name = 'articles/article_form.html'
    success_url = reverse_lazy('article-list')
```

10. *DeleteView:*

- Handles the deletion of an object.

Example:

```
from django.views.generic import DeleteView
from .models import Article
from django.urls import reverse_lazy
class ArticleDeleteView(DeleteView):
    model = Article
    template_name = 'articles/article_confirm_delete.html'
    success_url = reverse_lazy('article-list')
```

Advantages of Class-Based Views

- **Reusability:** CBVs allow you to create reusable components, reducing code duplication.
- **Extensibility:** You can easily extend existing views by overriding methods or using mixins.
- **Organization:** CBVs promote a more structured and organized codebase, especially for larger projects.
- **Built-in Functionality:** Generic views provide out-of-the-box solutions for common tasks, saving development time.

When to Use Class-Based Views

- **Complex Views:** Use CBVs for complex views that involve multiple actions, data processing, or reuse of logic.
- **Built-In Generic Views:** Use CBVs when you can leverage Django's generic views to handle common patterns like list views, detail views, and forms.
- **Large Projects:** In large projects with many views, CBVs help maintain a clean and organized code structure.

Class-Based Views in Django offer a powerful and flexible way to handle complex views and repetitive tasks

Comparison: Function-Based Views vs. Class-Based Views

Aspect	Function-Based Views (FBVs)	Class-Based Views (CBVs)
Simplicity	Simple and straightforward	More complex due to class and method structure
Control	Full control over request handling	Can override methods for specific behaviour
Reusability	Less reusable, requires manual code duplication	Highly reusable through inheritance and mixins
Flexibility	Highly flexible, no constraints	Flexible but within the confines of class structure

Learning Curve	Easier for beginners	Steeper learning curve, especially for new developers
Best Use Cases	Simple, one-off views	Complex views, repetitive patterns, and large projects

DJANGO TEMPLATES

- Django provides a convenient way to generate dynamic HTML pages by using its template system.
- A template consists of static parts of the desired HTML output as well as some special syntax describing how dynamic content will be inserted.
- Django template engine is used to separate the design from the python code and allows us to build dynamic web pages.
- **Definition:** A template in Django is a text file that defines the structure or layout of an HTML page, which includes placeholders for dynamic content.
- **Purpose:** Templates are used to separate the presentation layer (HTML) from the business logic in a web application. This ensures that the design and content management of a web page are separated from the underlying code.

Key Features of Django Templates:

- **Dynamic Content Rendering:** Templates allow you to generate dynamic HTML by inserting data from Django views into placeholders.
- **Template Language:** Django has its own template language that includes variables, tags, filters, and comments to help you manage content dynamically.
- **Inheritance:** Templates can inherit from other templates, allowing you to create a base layout that can be extended by other templates, promoting DRY (Don't Repeat Yourself) principles.

DJANGO TEMPLATE LANGUAGE COMPONENTS

a. Template Variables

- **Definition:** Variables are placeholders in templates that are replaced with values when the template is rendered.
- **Syntax:** Variables are enclosed within double curly braces `{{ }}`.

Example:

```
<h1>Welcome, {{ username }}!</h1>
```

Here, `{{ username }}` will be replaced with the actual username provided by the view.

b. Template Tags

- **Definition:** Tags are used for logic in templates, such as loops, conditionals, and including other templates. Tags are enclosed within `{% %}`.
- *Common Tags:*
 - **{% for %}:** Used for looping over a list.
 - **{% if %}:** Used for conditional statements.
 - **{% block %} and {% extends %}:** Used for template inheritance.

Example:

```
{% if user.is_authenticated %}
    <p>Hello, {{ user.username }}!</p>
{% else %}
    <p>Hello, Guest!</p>
{% endif %}
```

c. Template Filters

- **Definition:** Filters are used to modify the display of variables, like formatting dates, converting text to uppercase, etc.
- **Syntax:** Filters are applied using a pipe `|`.

Example:

```
<p>The current date is {{ date_today|date:"F d, Y" }}</p>
```

Here, the date filter formats the date_today variable to "Month Day, Year" format.

d. Template Comments

- **Definition:** Comments in Django templates are ignored by the template engine and are not rendered in the final HTML.
- **Syntax:** Comments are enclosed within `{# #}`.

Example:

```
{# This is a comment and will not appear in the HTML #}
```

TEMPLATE INHERITANCE**a. Purpose of Template Inheritance**

- **Definition:** Template inheritance allows you to define a base template (e.g., for the layout of a website) and then extend it in other templates.
- **Advantages:** This approach promotes reusability and consistency across the website.

b. Base Template

- A base template contains common elements like the header, footer, and navigation bar, which will be shared across multiple pages.

Example: base.html

```
<html>
<head>
  <title>{% block title %}My Site{% endblock %}</title>
</head>
<body>
  <header>
    <h1>Welcome to My Site</h1>
  </header>
  <nav>
    <!-- Navigation menu -->
  </nav>
  <main>
    {% block content %}{% endblock %}
  </main>
  <footer>
    <p>&copy; 2024 My Site</p>
  </footer>
</body>
</html>
```

c. Extending a Base Template

- Child templates can extend the base template and replace the content of the blocks defined in the base template.

Example: home.html

```
{% extends "base.html" %}
{% block title %}Home - My Site{% endblock %}
{% block content %}
<h2>Home Page</h2>
<p>This is the home page content.</p>
{% endblock %}
```

d. Block Tag

- **Purpose:** The `{% block %}` tag is used to define areas in a template that can be overridden by child templates.

- *Syntax:*

```
{% block block_name %}Default content{% endblock %}
```

TEMPLATE CONTEXT

- **Definition:** The context in Django refers to a dictionary of data that is passed from the view to the template. This data can include variables, lists, or any other data needed to render the template.
- **Rendering a Template:** The render function in Django combines the template with the context and returns an HTTP response with the rendered HTML.

Example:

```
def my_view(request):
```

```
    context = {
```

```
        'username': 'JohnDoe',
```

```
        'date_today': datetime.date.today(),
```

```
    }
```

```
    return render(request, 'my_template.html', context)
```

- **Accessing Context in Templates:** In the template, you can access context variables using `{{ variable_name }}`.

Example:

```
<p>Username: {{ username }}</p>
```

```
<p>Today's Date: {{ date_today }}</p>
```

TEMPLATE LOADERS

- **Definition:** Template loaders are responsible for finding and loading templates from the file system or other sources like databases.
- *Default Loaders:*
 - ✓ **Filesystem Loader:** Loads templates from the directories specified in `TEMPLATES['DIRS']`.
 - ✓ **App Directories Loader:** Looks for templates within each app's templates directory.

TEMPLATE INCLUSIONS

- **Purpose:** The `{% include %}` tag allows you to include the content of another template within a template.

Example:

```
{% include "partials/header.html" %}
```

```
<p>Main content here.</p>
```

```
{% include "partials/footer.html" %}
```

DJANGO FORMS- SIMPLE REGISTRATION FORM CREATION

To display a student registration form with detailed steps, we'll set up a Django project and app, create a form, set up a view to render the form, and create the necessary template.

1. Set Up Your Django Project and App

a. Create a Django Project:

1. Open your terminal or command prompt.
2. Navigate to the directory where you want to create your Django project.
3. Run the following command to create a new Django project:
`django-admin startproject myproject`
4. Navigate into your project directory:
`cd myproject`

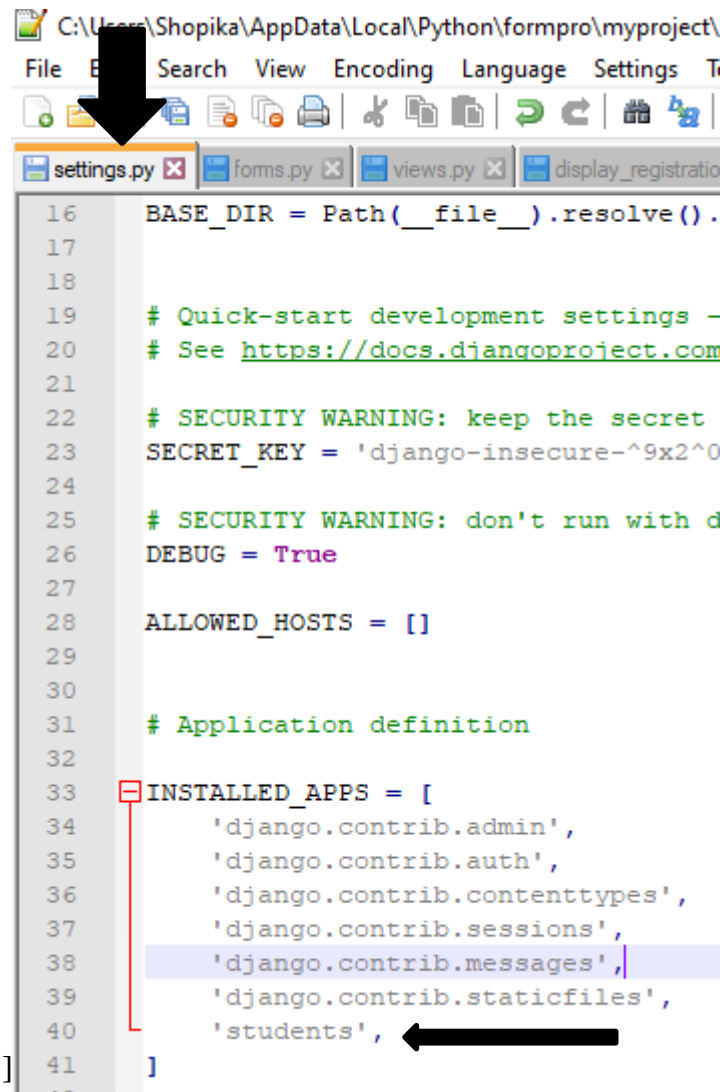
b. Create a Django App:

1. Run the following command to create a new app within your project:
`python manage.py startapp students`

```
(venv) C:\Users\Shopika\AppData\Local\Python\formpro>django-admin startproject myproject
(venv) C:\Users\Shopika\AppData\Local\Python\formpro>
(venv) C:\Users\Shopika\AppData\Local\Python\formpro>cd myproject
(venv) C:\Users\Shopika\AppData\Local\Python\formpro\myproject>python manage.py startapp students
```

2. Add the students app to your project's settings. Open myproject/settings.py and add 'students' to the INSTALLED_APPS list:

```
INSTALLED_APPS = [
    ...
    'students',
```



```

16 BASE_DIR = Path(__file__).resolve().
17
18
19 # Quick-start development settings -
20 # See https://docs.djangoproject.com
21
22 # SECURITY WARNING: keep the secret
23 SECRET_KEY = 'django-insecure-^9x2^0
24
25 # SECURITY WARNING: don't run with d
26 DEBUG = True
27
28 ALLOWED_HOSTS = []
29
30
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'students',
41 ]

```

2. Create the Form

a. Define the Form in forms.py:

1. Inside **the students app directory**, create a file named **forms.py** if it doesn't already exist.
2. Define the form class for student registration:

students/forms.py

from django import forms

class StudentRegistrationForm(forms.Form):

first_name = forms.CharField(max_length=100, label='First Name')

last_name = forms.CharField(max_length=100, label='Last Name')

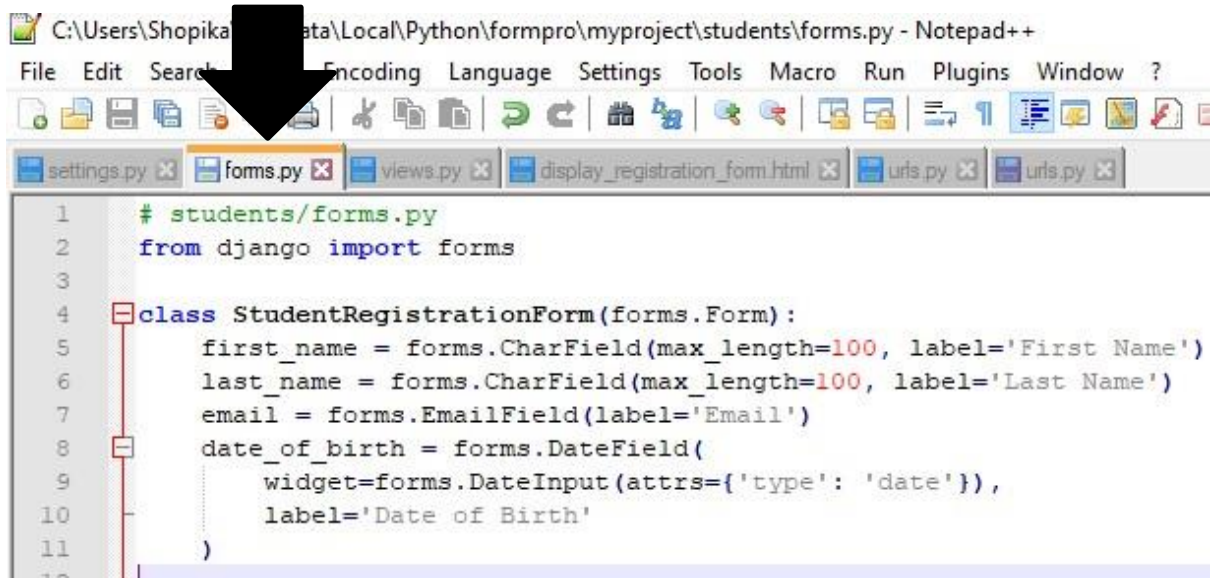
email = forms.EmailField(label='Email')

date_of_birth = forms.DateField(

widget=forms.DateInput(attrs={'type': 'date'}),

label='Date of Birth'

)



```

1  # students/forms.py
2  from django import forms
3
4  class StudentRegistrationForm(forms.Form):
5      first_name = forms.CharField(max_length=100, label='First Name')
6      last_name = forms.CharField(max_length=100, label='Last Name')
7      email = forms.EmailField(label='Email')
8      date_of_birth = forms.DateField(
9          widget=forms.DateInput(attrs={'type': 'date'}),
10         label='Date of Birth'
11     )
12

```

3. Create the View

a. Define the View in views.py:

1. Open students/views.py.
2. Create a view function to render the form:

students/views.py

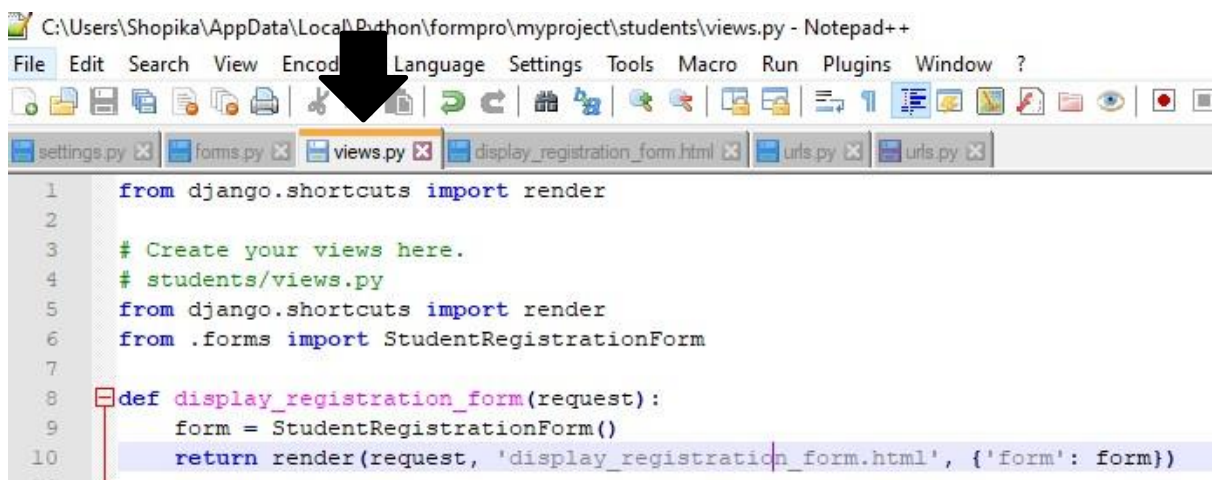
from django.shortcuts import render

from .forms import StudentRegistrationForm

def display_registration_form(request):

 form = StudentRegistrationForm()

 return render(request, 'display_registration_form.html', {'form': form})



```

1  from django.shortcuts import render
2
3  # Create your views here.
4  # students/views.py
5  from django.shortcuts import render
6  from .forms import StudentRegistrationForm
7
8  def display_registration_form(request):
9      form = StudentRegistrationForm()
10     return render(request, 'display_registration_form.html', {'form': form})
11

```

4. Create the Template

a. Create the Template File:

1. Inside the students app directory, create a folder named templates.

Local Disk (C:) > Users > Shopika > AppData > Local > Python > formpro > myproject > students

Name	Date modified	Type	Size
__pycache__	11-08-2024 22:36	File folder	
migrations	11-08-2024 22:30	File folder	
templates	11-08-2024 22:24	File folder	
__init__	11-08-2024 22:19	Python File	0 KB
admin	11-08-2024 22:19	Python File	1 KB
apps	11-08-2024 22:19	Python File	1 KB
forms	11-08-2024 22:22	Python File	1 KB
models	11-08-2024 22:36	Python File	1 KB
tests	11-08-2024 22:19	Python File	1 KB
urls	11-08-2024 22:28	Python File	1 KB
views	11-08-2024 22:30	Python File	1 KB

2. Within the templates folder, create a file named display_registration_form.html.

Local Disk (C:) > Users > Shopika > AppData > Local > Python > formpro > myproject > students > templates

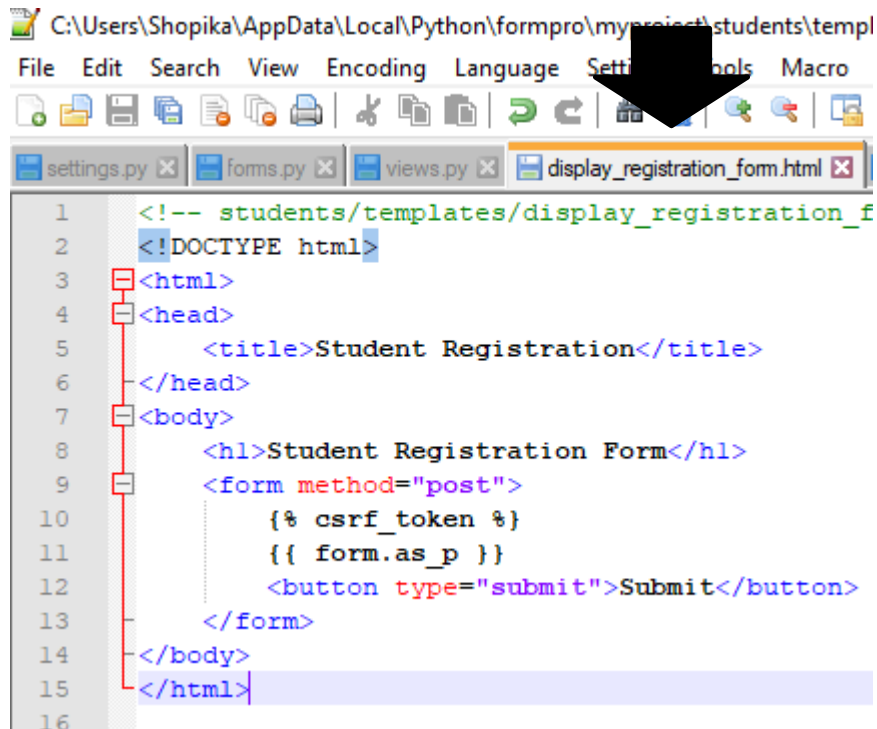
Name	Date modified	Type	Size
display_registration_form	11-08-2024 22:24	HTML Document	1 KB

3. Add the following HTML code to the file:

```
<!-- students/templates/display_registration_form.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
</head>
<body>
  <h1>Student Registration Form</h1>
  <form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
  </form>
```

```
</body>
```

```
</html>
```



```

1  <!-- students/templates/display_registration_f
2  <!DOCTYPE html>
3  <html>
4  <head>
5      <title>Student Registration</title>
6  </head>
7  <body>
8      <h1>Student Registration Form</h1>
9      <form method="post">
10         {% csrf_token %}
11         {{ form.as_p }}
12         <button type="submit">Submit</button>
13     </form>
14 </body>
15 </html>
16

```

5. Set Up URL Routing

a. Configure URLs in urls.py:

1. Create or edit the urls.py file in the students app directory:

```
# students/urls.py
```

```
from django.urls import path
```

```
from .views import display_registration_form
```

```
urlpatterns = [
```

```
    path('register/', display_registration_form, name='display_registration_form'),
```

```
]
```



Local Disk (C:) > Users > Shopika > AppData > Local > Python > formpro > myproject > students

Name	Date modified	Type	Size
__pycache__	11-08-2024 22:36	File folder	
migrations	11-08-2024 22:30	File folder	
templates	11-08-2024 22:24	File folder	
__init__	11-08-2024 22:19	Python File	0 KB
admin	11-08-2024 22:19	Python File	1 KB
apps	11-08-2024 22:19	Python File	1 KB
forms	11-08-2024 22:22	Python File	1 KB
models	11-08-2024 22:36	Python File	1 KB
tests	11-08-2024 22:19	Python File	1 KB
urls	11-08-2024 22:28	Python File	1 KB
views	11-08-2024 22:30	Python File	1 KB

C:\Users\Shopika\AppData\Local\Python\formpro\myproject\students\urls.py - Notepad++

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

settings.py forms.py views.py display_registration_form.html urls.py

```

1  """
2  URL configuration for myproject project.
3
4  The `urlpatterns` list routes URLs to views. For more information please see:
5  https://docs.djangoproject.com/en/5.1/topics/http/urls/
6  Examples:
7  Function views
8      1. Add an import:  from my_app import views
9      2. Add a URL to urlpatterns:  path('', views.home, name='home')
10 Class-based views
11     1. Add an import:  from other_app.views import Home
12     2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
13 Including another URLconf
14     1. Import the include() function: from django.urls import include, path
15     2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
16 """
17
18 from django.urls import path
19
20 from .views import display_registration_form
21
22 urlpatterns = [
23     path('register/', display_registration_form, name='display_registration_form'),
24 ]

```

2. Include the students app URLs in the main project's URL configuration:

Open myproject/urls.py and add an import statement for include, then include the students URLs:

myproject/urls.py

from django.contrib import admin

from django.urls import path, include

urlpatterns = [

 path('admin/', admin.site.urls),




```
path('students/', include('students.urls')),
```

```

1  """
2  URL configuration for myproject project.
3
4  The 'urlpatterns' list routes URLs to views. For more information see the Django documentation:
5  https://docs.djangoproject.com/en/5.1/topics/http/urls/
6  Examples:
7  Function views
8      1. Add an import: from my_app import views
9      2. Add a URL to urlpatterns: path('', views.home, name='home')
10 Class-based views
11     1. Add an import: from other_app.views import Home
12     2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13 Including another URLconf
14     1. Import the include() function: from django.urls import include
15     2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16 """
17
18 # myproject/urls.py
19 from django.contrib import admin
20 from django.urls import path, include
21
22 urlpatterns = [
23     path('admin/', admin.site.urls),
24     path('students/', include('students.urls')),
25 ]

```

6. Run the Development Server

1. In your terminal, navigate to the root directory of your Django project (myproject).

2. Run the development server:

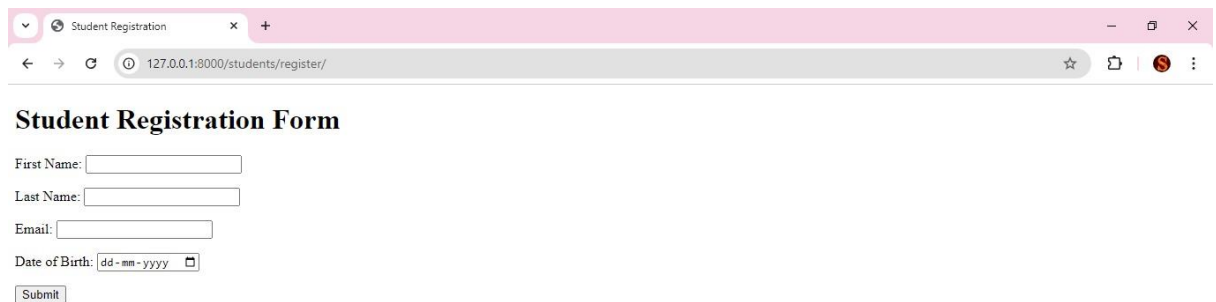
```
python manage.py runserver
```

```
(venv) C:\Users\Shopika\AppData\Local\Python\formpro\myproject>
(venv) C:\Users\Shopika\AppData\Local\Python\formpro\myproject>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the
auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
August 11, 2024 - 22:36:40
Django version 5.1, using settings 'myproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

3. Open your web browser and go to **http://127.0.0.1:8000/students/register/** to see the student registration form.



The screenshot shows a web browser window with the title "Student Registration". The address bar displays "127.0.0.1:8000/students/register/". The page content includes the heading "Student Registration Form" and a registration form with the following fields:

- First Name:
- Last Name:
- Email:
- Date of Birth: (with a calendar icon)

At the bottom of the form is a "Submit" button.

Summary

1. **Set Up Project and App:** Create a Django project and app, then add the app to the project's settings.
2. **Create Form:** Define a form in forms.py with student registration fields.
3. **Create View:** Write a view function in views.py to render the form.
4. **Create Template:** Design a template to display the form.
5. **Set Up URL Routing:** Configure URL routing to map the view to a URL.
6. **Run Server:** Start the Django development server and view the form in your browser.

This setup allows you to display a student registration form with fields like First Name, Last Name, Email, and Date of Birth, without handling form submissions.